

数 字 逻 辑

丁 贤 庆

ahhfdxq@163.com

自我介绍

- 1、系统结构与情感计算
- 2、研究方向：嵌入式系统设计
- 3、有20多年的教学经验。讲授过：数字逻辑电路、计算机组成原理、计算机系统结构、嵌入式系统、单片机设计、C程序设计、面向对象程序设计等多门课程。
- 4、有疑难问题，大家可以通过QQ私聊我，只要我有空，基本上都会秒回。如果当时没空，也会在有空时进行回复。

希望各位同学

- 1、尽量不缺课
- 2、必须认真看书、复习
- 3、完成一定数量的习题（每次课布置三、五个必做题，每个单周的第一次课（周二/周三）收一次作业本），也就是说：第3周，第5周，第7周收作业本。第8周的作业交电子稿，要求在期末考试前3天交给学委，可以将作业本上的答题拍照放到WORD文档里。也可以用ipad直接写文档。文档也可以是pdf格式。文档命名格式： 学号_姓名_数字逻辑第8周作业。

要求

一、上课会听课。（课前预习，课后复习。）

多听，少记。多想，多问。

二、按时完成作业。 有独立的作业本

严禁抄袭！！！独立完成。

三、课堂考核方式。

每节课随机点名。

重点内容会找同学在线演算。

上课时 would 随时找同学回答问题。

第4周，第6周，第8周会有课堂10分钟测验时间。

四、实验

独立完成实验（4次），要写实验报告（存档）。

五、需要记忆一些英语专业词汇。对以后科研有帮助。

Grades (成绩组成)

学时：32 +8 (2.5学分)。其中上课8周，共32学时。

成绩比例：

- 作业 (Homeworks) — 20%
- 实验 (含实验报告) (experiment and Lab reports) — 20%
- 课堂考核 (含平时点名和随堂小测) (Quizzes in class)— 20%
- 期末考试 (Final Exam, Closed book) — 40%
- 期末卷面成绩低于45分的，学校政策是：直接挂课，需要补考。

实验安排

- 实验安排：
- 根据课表中的空闲时间来安排, 会提前通知的, 占成绩。请各班的学委将本班的空闲时间表发给我（填写**第13周到第18周**可以安排实验的时间），然后将表格发给我的QQ，或者我的邮箱ahhfdxq@163.com（空表格从QQ群共享文件夹里下载。将表格里第13周~第18周空闲时间段标明，以便我在空闲时间段内安排实验。**4次实验，每周1次**）。
- （要和实验室预约）

答 疑

计算机1、2、3班加入QQ群：607351585

计算机4、5班加入QQ群：613639520

物联网1、2班加入QQ群：671530461

群号的后8位是加群密码，密码有效期是2周。

欢迎通过电子邮件提问

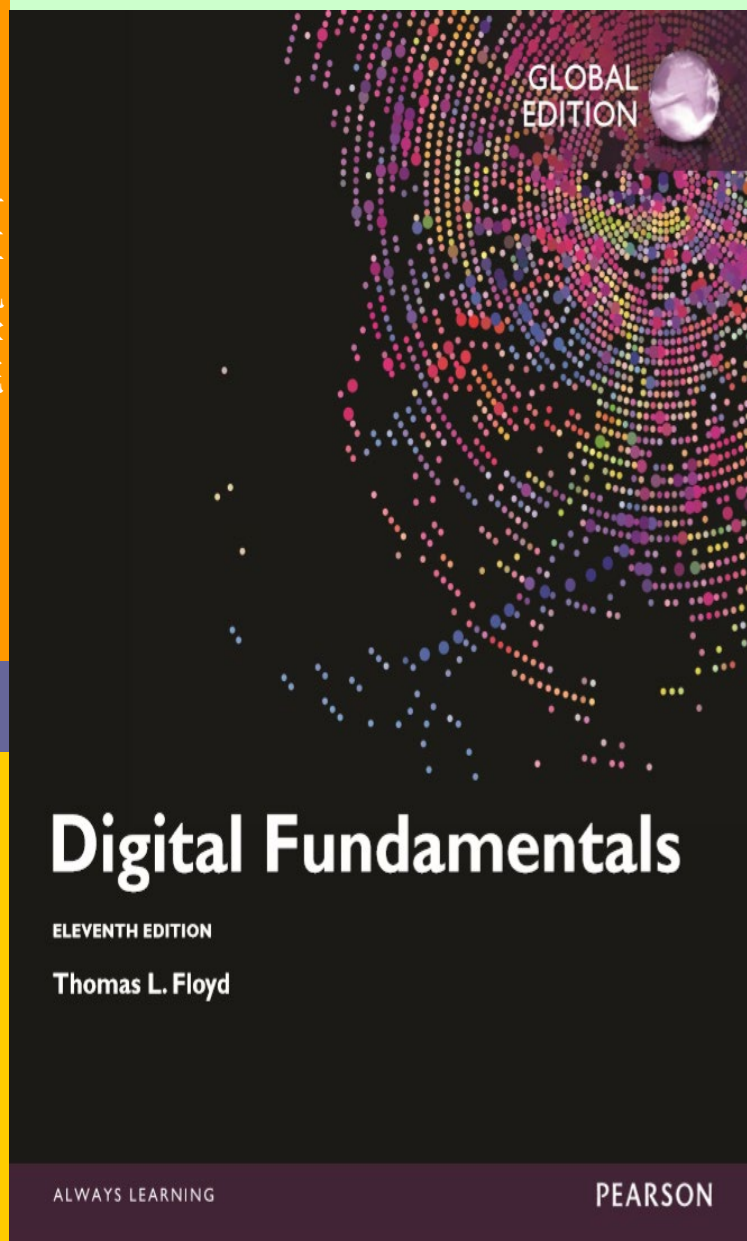
我的E-mail: ahhfdxq@163.com

祝同学们身体健康！学业有成！

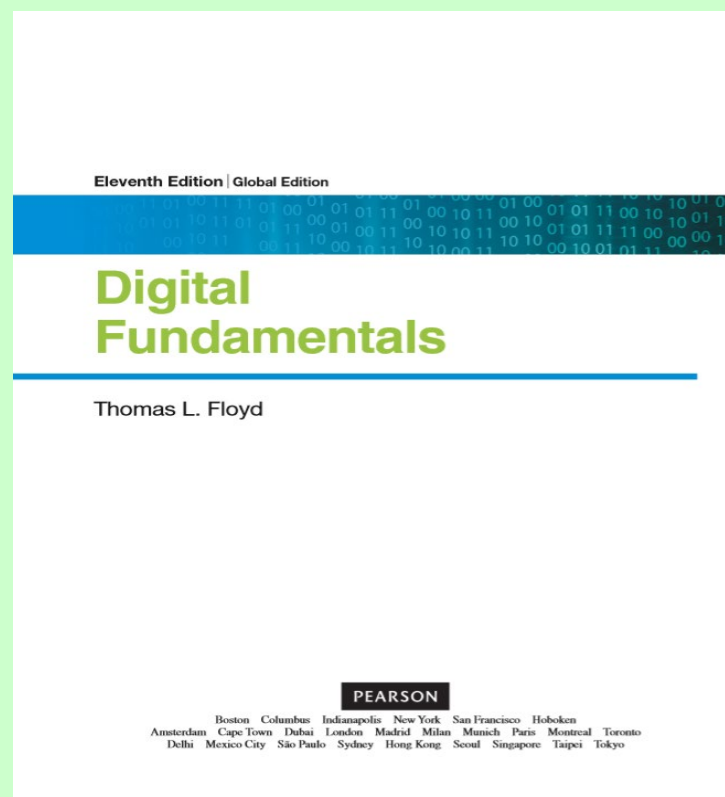


- 主教材（Textbook:）：
- 《电子技术基础—数字部分》 康华光主编
华中科技大学出版社
第六版/第七版

只讲本书的前**6**章，
期末考试也只考前**6**章。



- 参考教材（References:）：
- 《**Digital Fundamentals**》
Thomas L. Floyd 主编
Pearson Education Intemational 出版



学堂在线教学视频和测试

合肥工业大学计算机学院刘军老师录制的教学视频和考试样题。

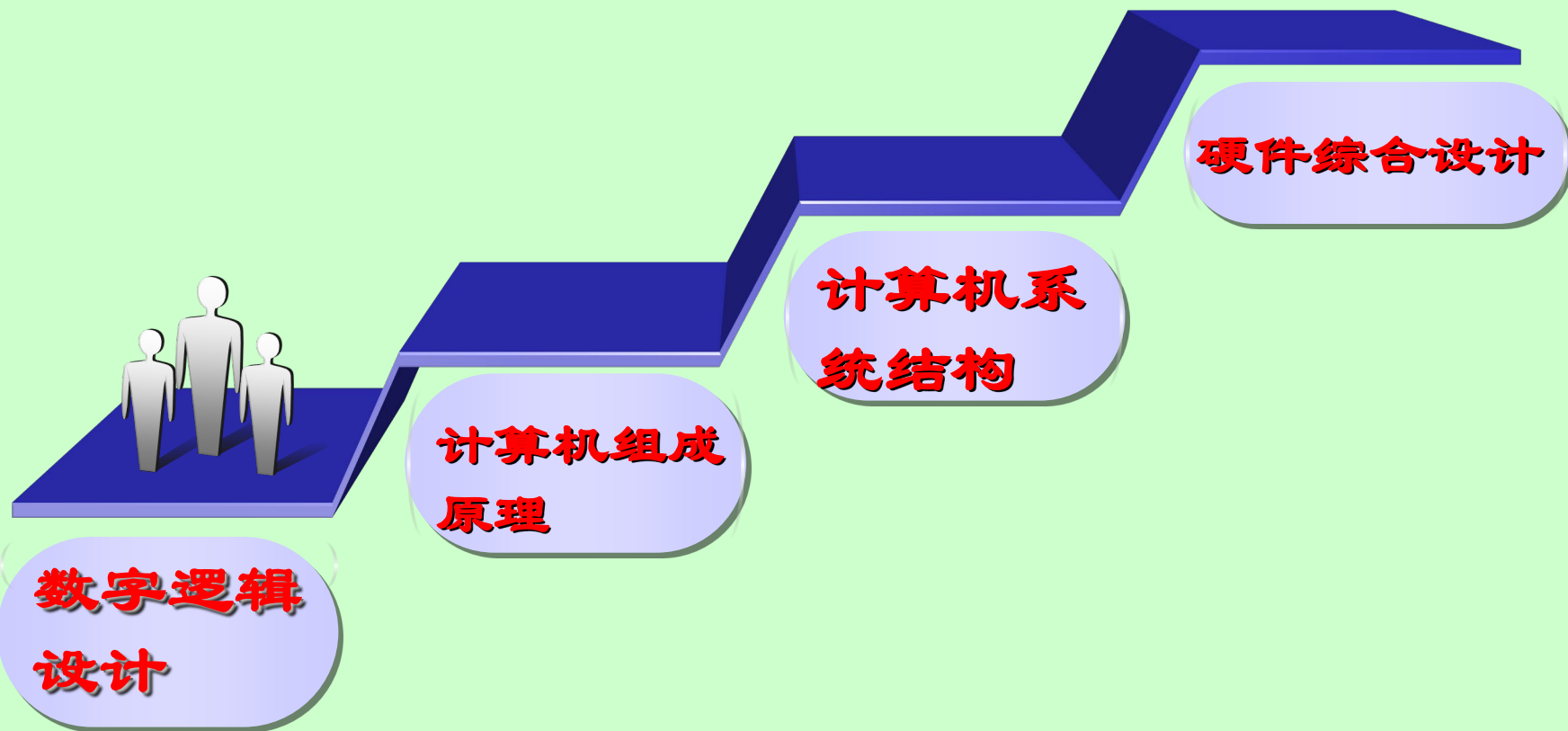
[https://www.xuetangx.com/course/hfut08091005182/12426819?
channel=i.area.manual_search](https://www.xuetangx.com/course/hfut08091005182/12426819?channel=i.area.manual_search)

该教学视频是免费学习的。用 QQ号登陆就可以免费注册和学习。

我们的期末考试的难度和该网址里面考试样题的难度相当。

Relation with other courses

■ 与后续课程 关系



数字电路应用广泛



电视技术



雷达技术



航空航天



计算机、自动控制

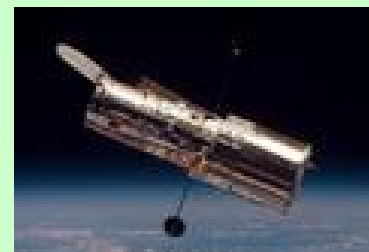


通信技术

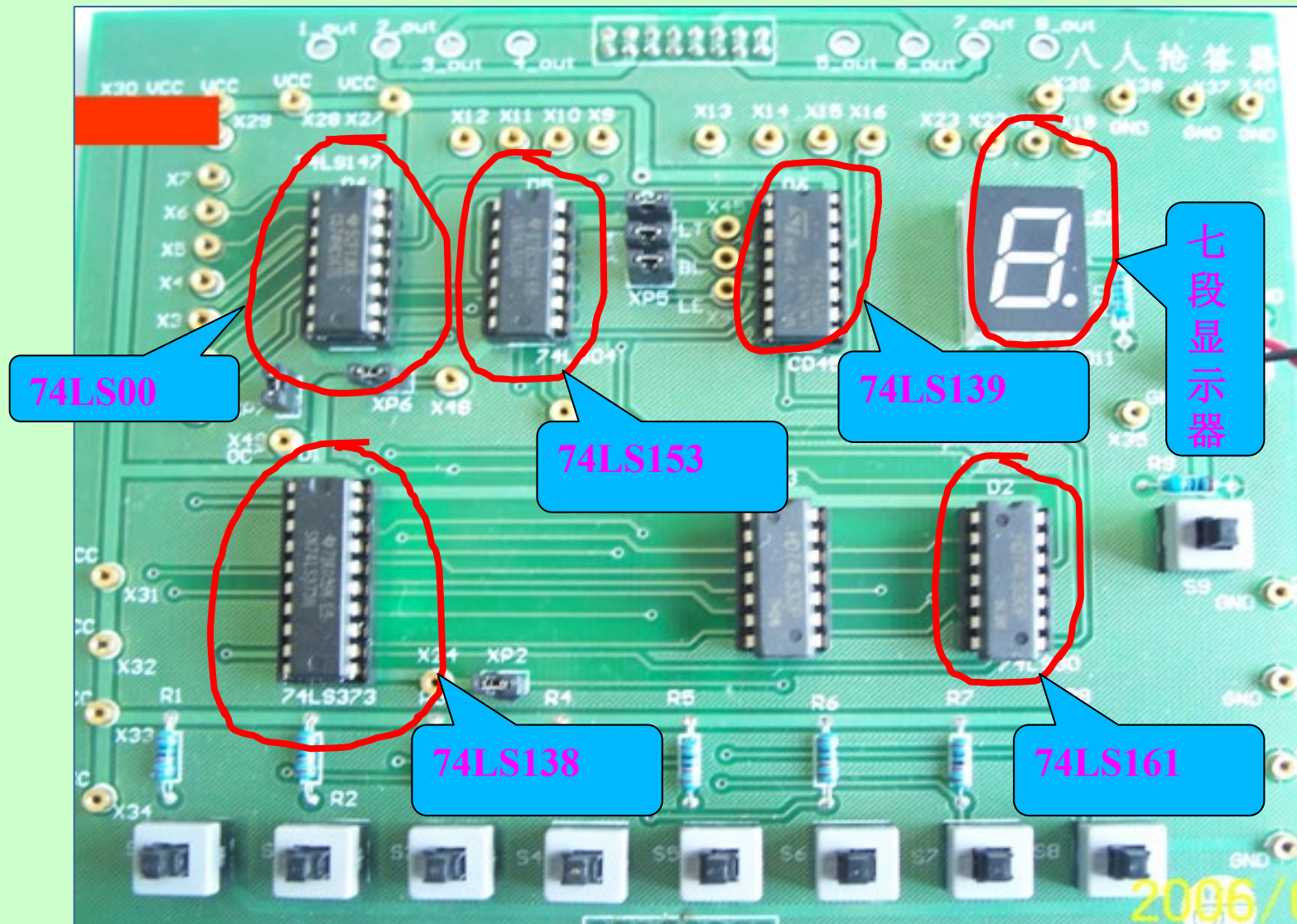
21世纪是信息数字化的时代，“数字逻辑”是数字技术的基础。



数字逻辑电路的应用



以下几款芯片的结构和工作特性要求掌握



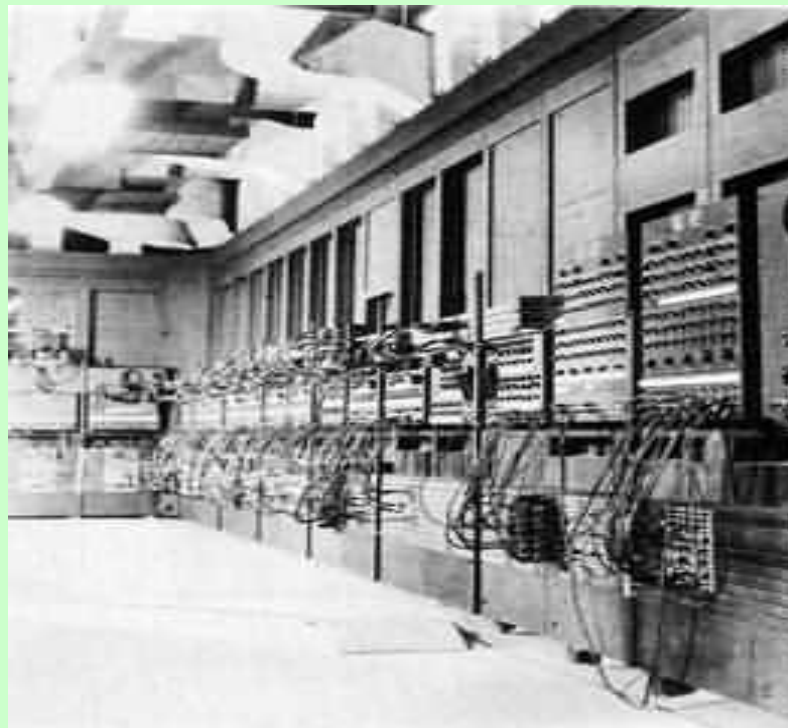
数字电路技术的发展历程：

电子管→晶体管→集成电路

电子管时代

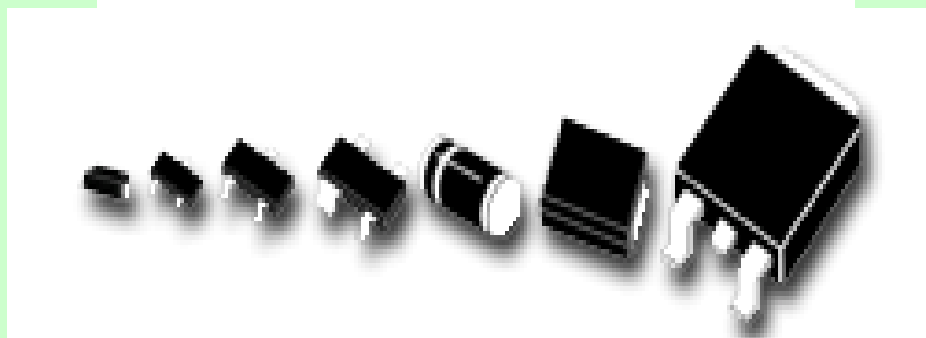


电子管



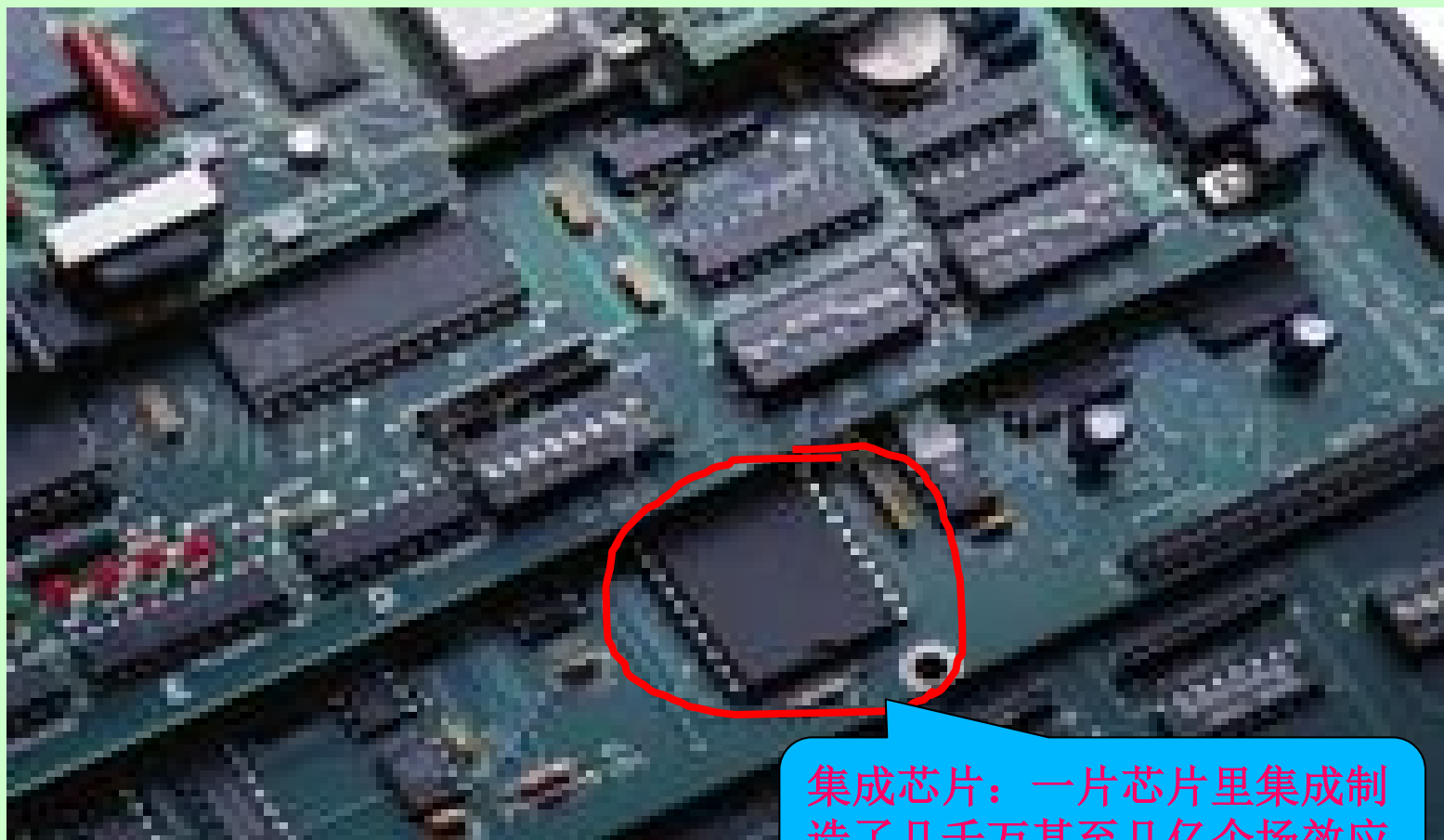
世界第一台电子计算机

晶体管时代



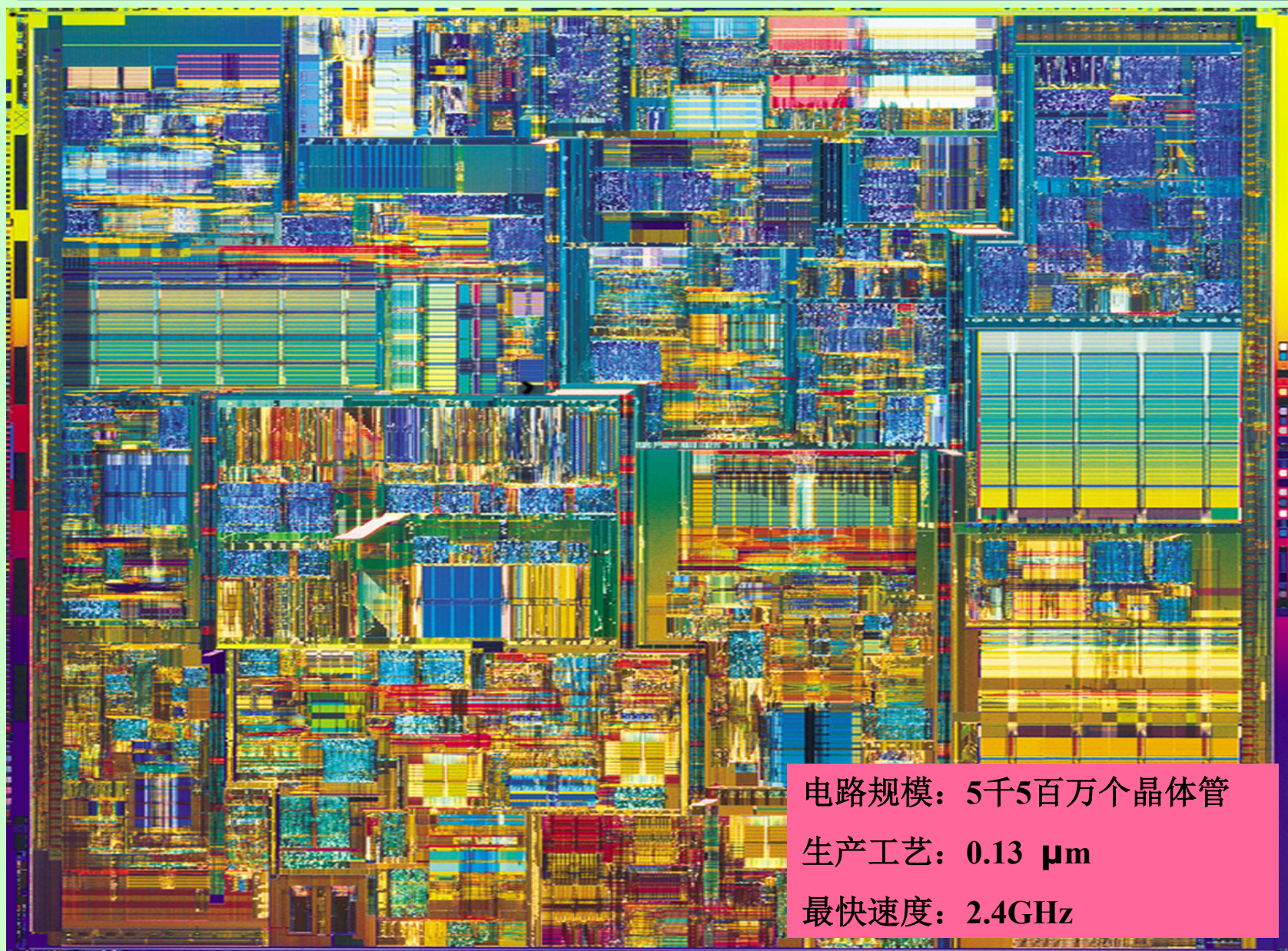
晶体管（三极管）

集成电路时代



集成芯片：一片芯片里集成制造了几千万甚至几亿个场效应管（晶体管）。

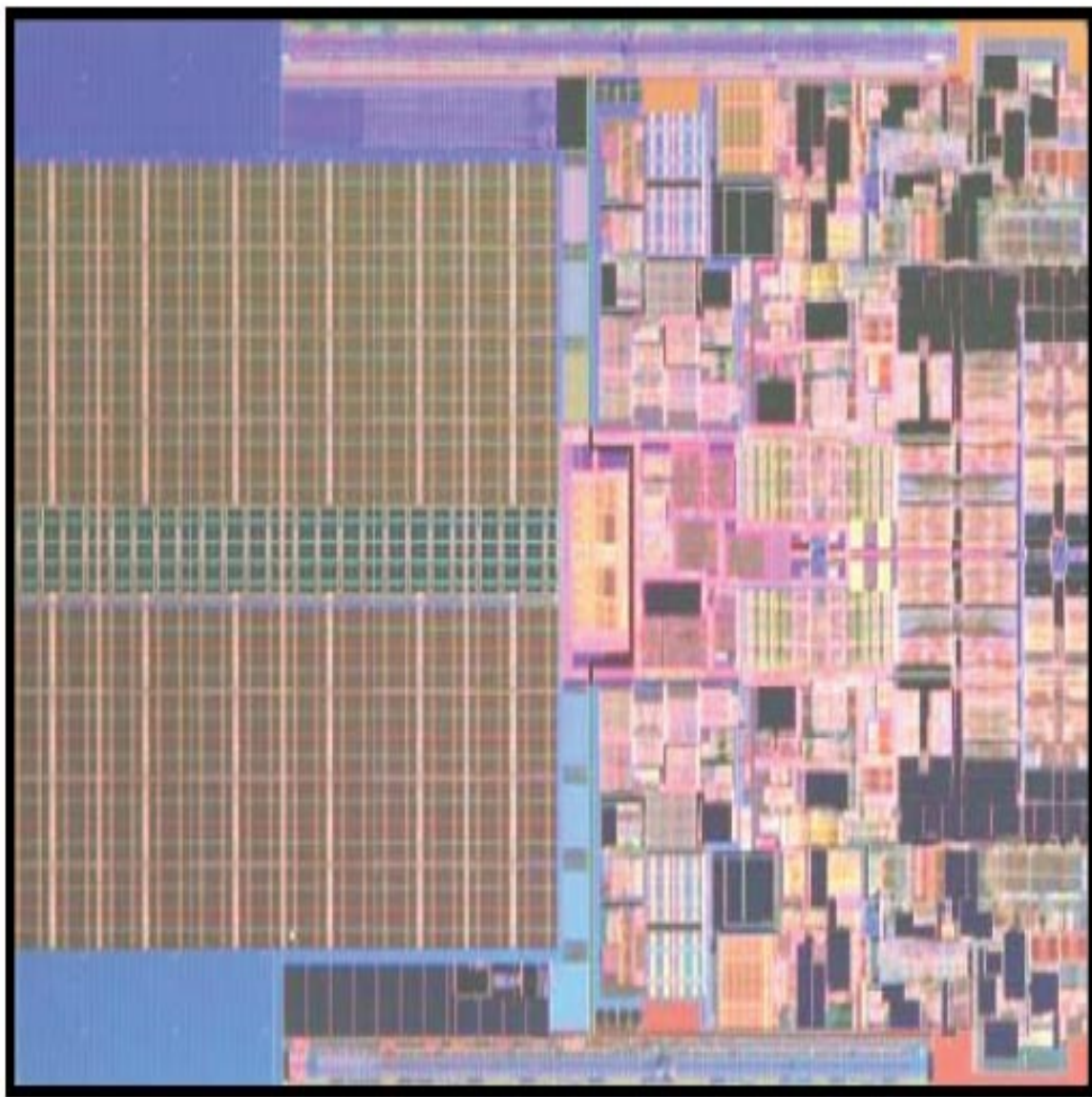
❖ 2002年Intel 公司研制出CPU—Pentium 4内部结构（一块指甲盖大的芯片）



电路规模：5千5百万个晶体管

生产工艺：0.13 μm

最快速度：2.4GHz



2002年8月13日，英特尔开始**90nm**制程的突破，业内首次在生产中采用应变硅；

2005年顺利过渡到了**65nm**工艺。

2007年英特尔推出**45nm**正式量产工艺。

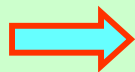
现在手机芯片已经达到了**5nm**的制作工艺。

芯片内导线越来越细。

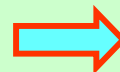
电子技术的发展很大程度上反映在元器件的发展上。从电子管→半导体管→集成电路



1904年
电子管问世



1947年
晶体管诞生



1958年集成电
路研制成功

第一章

逻辑电路理论基础

丁 贤 庆

ahhfdxq@163.com

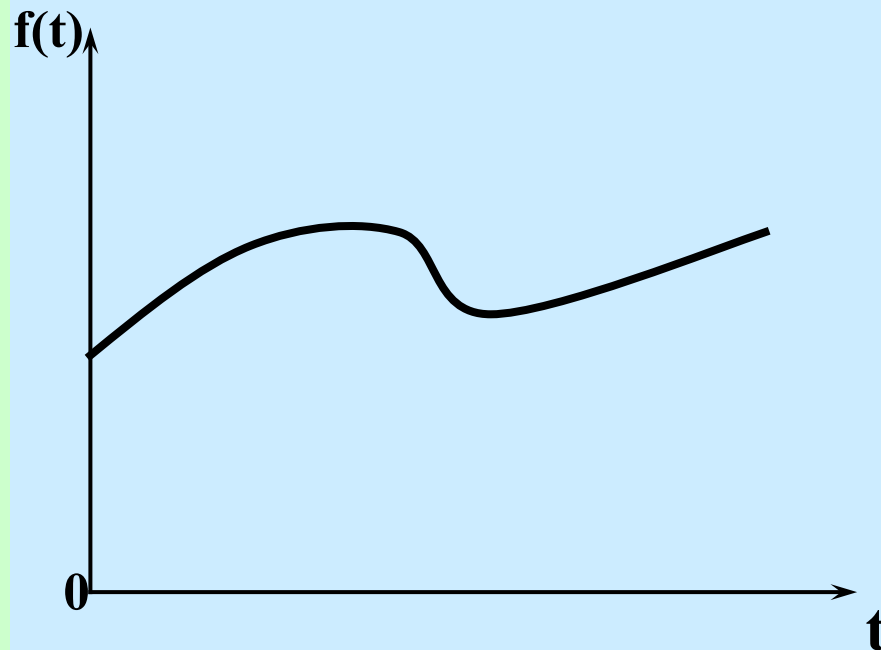
Home work (P38)

- ☞ **1.2.1**
- ☞ **1.2.4(2)**
- ☞ **1.2.6(4)**
- ☞ **1.2.8(4)**
- ☞ **1.2.11**

几个基本概念

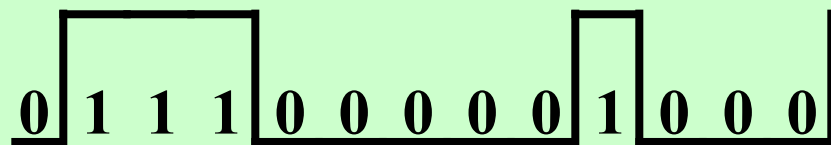
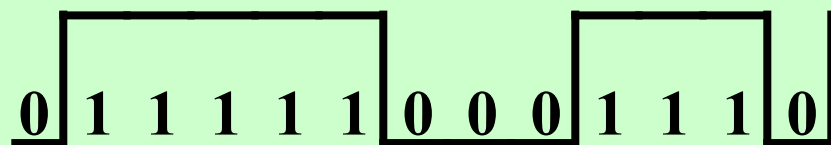
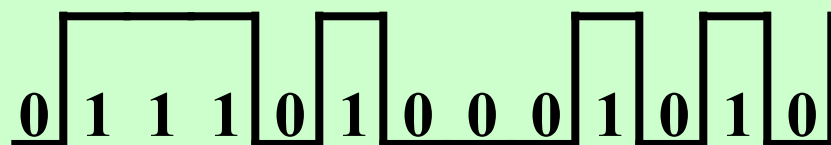
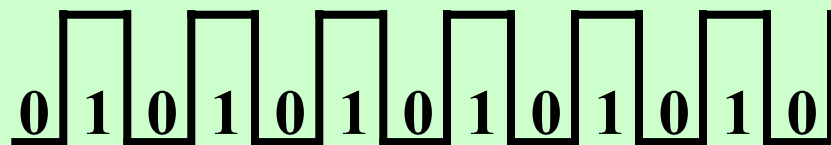
- 在电子技术中，被传递、加工和处理的信号可以分为两大类：模拟信号和数字信号
- **模拟信号**：在时间上和幅度上都是**连续变化的信号**，在一定动态范围内可取任意值。例如正弦波信号、心电信号等。
- **数字信号**：在时间和幅度上是**离散的**，不连续的信号。

模拟信号：在时间和幅度上都是连续变化的，在一定动态范围内可取任意值。



数字信号:在时间和幅值上都是**离散的(不连续)**信号。

数字信号只存在**高低两种不同的电平**。



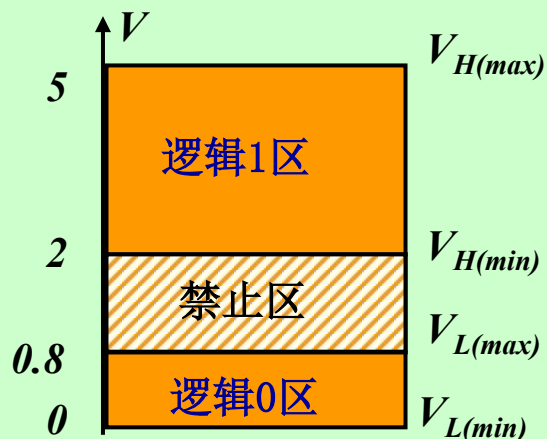
实际生活中相互对立的两种状态，例如：

信号的有与无
电平的高与低
开关的通与断
事情的真与假

都可以用1和0来表示。

用来表示 1 和 0 的电平

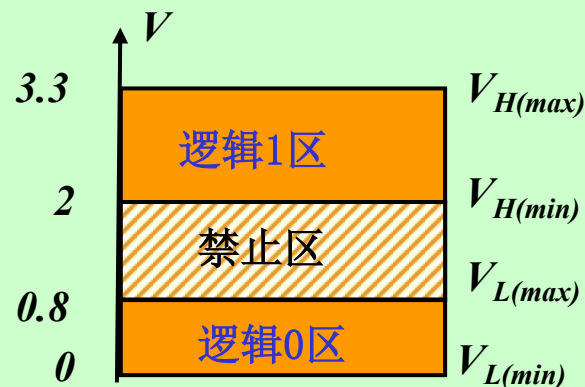
逻辑电平



TTL电路

Transistor-Transistor Logic

晶体管晶体管逻辑电路



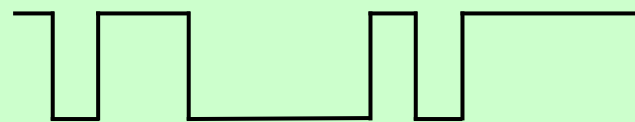
CMOS电路

Complementary metal-oxide-semiconductor

互补金属氧化物半导体

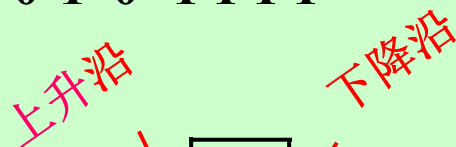
将数字量的两个状态 1 和 0 用波形表示

数字波形

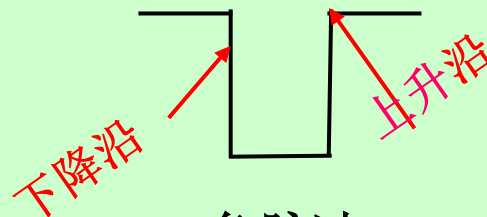


1 0 1 1 0 0 0 0 1 0 1 1 1 1

理想脉冲波形

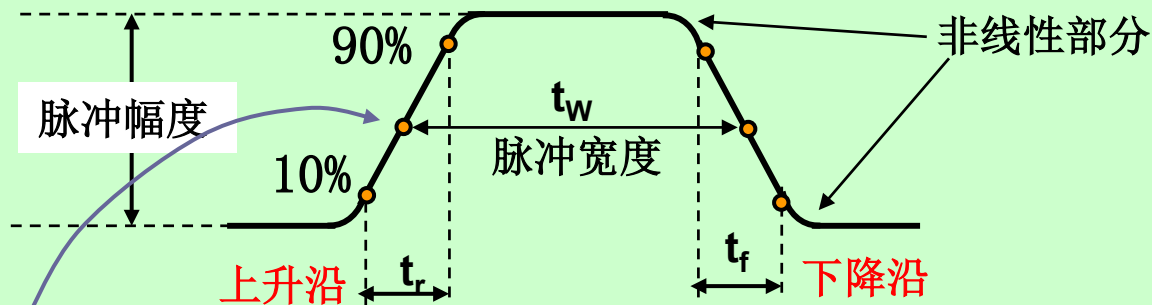


正脉冲



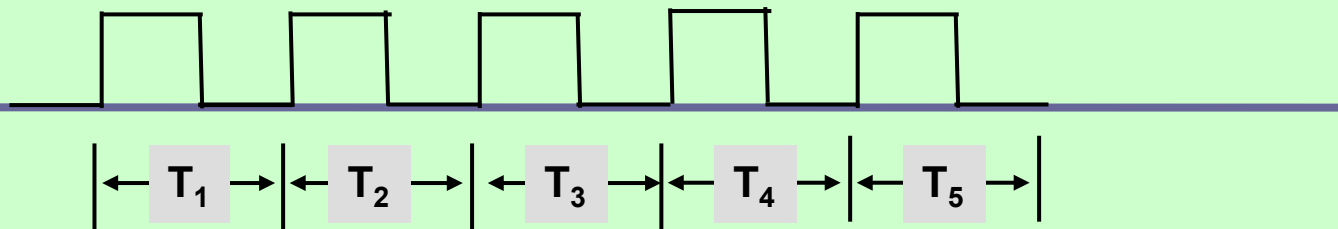
负脉冲

非理想脉冲波形



从低电平到高电平需要过程

周期性波形

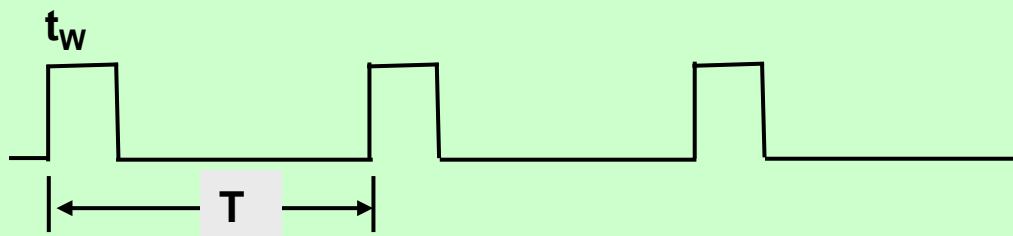


特点：波形在固定的时间间隔内重复。

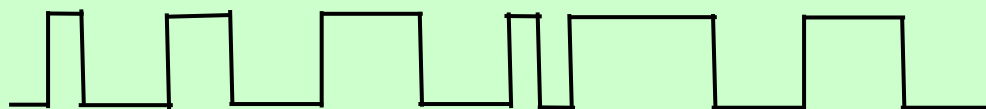
三个重要参数

$$\begin{cases} \text{脉冲周期 } T \\ \text{脉冲频率 } f \\ \text{频宽比 } D \text{ (占空比)} \end{cases} \quad \begin{aligned} f &= \frac{1}{T} \\ D &= \left(\frac{t_w}{T} \right) \times 100\% \end{aligned}$$

$$\text{周期 } T = T_1 = T_2 = T_3 = T_4 = T_5$$



非周期性波形

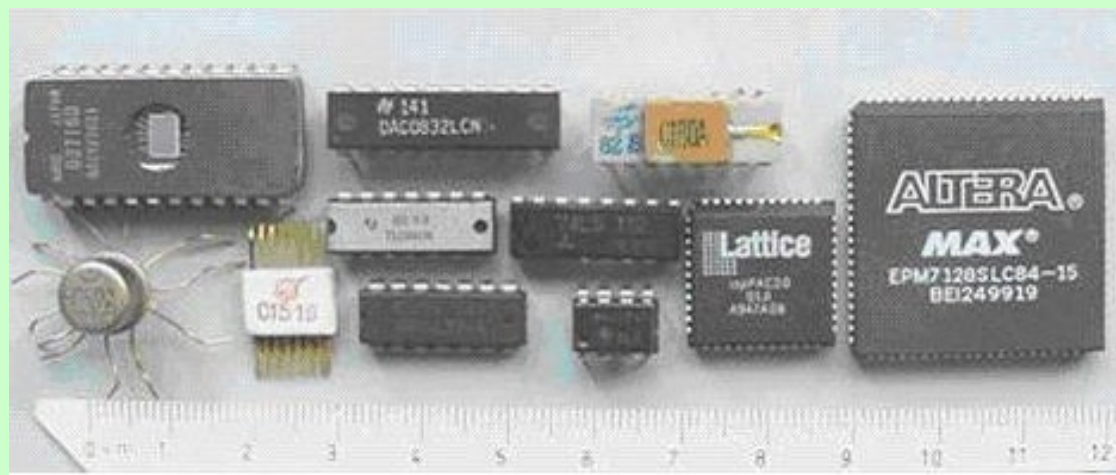


特点：波形不在固定的时间间隔内重复。

- **模拟电路：**工作信号为模拟信号的电路。
- **数字电路：**工作信号为数字信号的电路。
- **本课程研究的对象：**数字电路研究的对象是数字电路的**输出与输入之间的因果关系**，也就是说研究电路的**逻辑关系**。

- **数字系统：**用数字电路构成的实际工程系统统称为数字系统。
- **集成电路：**将多个电子电路器件做在一块**芯片**上的电路称为集成电路。

- 数字集成电路分类：小规模集成电路（SSI）、中规模集成电路（MSI）、大规模集成电路（LSI）、超大规模集成电路（VLSI）。



数字电路的特点

1. 数字电路中所有的半导体器件是工作在饱和区和截止区，而放大区只是作为一个短暂的过度过程。
2. 在数字电路中研究输入与输出之间的逻辑关系
3. 主要优点：抗干扰能力强，工作可靠性高，便于高度集成化



Contents

-  1.1 数制与编码.....●
-  1.2 逻辑函数.....●
-  1.3 布尔代数.....●
-  1.4 卡诺图.....●
-  1.5 集成门电路的外特性.....●

1.1 数制与编码

1.1.1 进位计数制

数码：由数字符号构成且表示物理量大小的数字和数字组合。

进位计数制（简称数制）：多位数码中每一位的构成方法，以及从低位到高位进制规则。

了解进位计数制的两个概念：**进位基数**和**数位的权值**（位权）。

进位基数：在一个数位上，规定使用的数码符号的个数叫该进位计数制的进位基数或进位模数，记作 R 。例如十进制，每个数位规定使用的数码符号为0, 1, 2, ..., 9, 共10个，故其进位基数 $R=10$ 。

数位的权值：某个数位所表征的数值，称为该数位的权值，简称“权”。各个数位的权值均可表示成 R^i 的形式，其中 R 是进位基数， i 是各数位的序号。

i 按如下方法确定：整数部分，以小数点为起点，自右向左依次为0, 1, 2, ..., $n-1$ ；小数部分，以小数点为起点，自左向右依次为-1, -2, ..., $-m$ 。 n 是整数部分的位数， m 是小数部分的位数。

某个数位上的数码 a_i 所表示的数值等于数码 a_i 与该位的权值 R^i 的乘积。所以， R 进制的数

$$(N)_R = a_{n-1}a_{n-2}\cdots a_2a_1a_0 \cdot a_{-1}a_{-2}\cdots a_{-m}$$

又可以写成如下多项式的形式：

$$\begin{aligned}(N)_R &= a_{n-1}R^{n-1} + a_{n-2}R^{n-2} + \cdots + a_2R^2 + a_1R^1 + a_0R^0 \\ &+ a_{-1}R^{-1} + a_{-2}R^{-2} + \cdots + a_{-m}R^{-m} \\ &= \sum_{i=-m}^{n-1} a_i R^i\end{aligned}$$

1.1.2 几种常用的进位计数制

1. 十进制

数字符号（系数）：0、1、2、3、4、5、6、7、8、9

计数规则：逢十进一

基数：10

各位的权值为 10^i ， i 是各数位的序号。

十进制数用下标“D”表示，也可省略。例如：

$$(368.258)_D = 3 \times 10^2 + 6 \times 10^1 + 8 \times 10^0 + 2 \times 10^{-1} + 5 \times 10^{-2} + 8 \times 10^{-3}$$

$$(N)_{10} = \sum_{i=-m}^{n-1} k_i \times 10^i$$

十进制数人们最熟悉，但机器实现起来困难。

2. 二进制

数字符号：0、1

计数规则：逢二进一

基数：2

权： 2^i ， i 是各数位的序号

二进制数用下标“B”表示。例如：

$$(1011.01)_B = 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 + 0 \times 2^{-1} + 1 \times 2^{-2}$$

$$(N)_2 = \sum_{i=-m}^{n-1} k_i \times 2^i$$

二进制数由于只需两个状态，机器实现容易，因而二进制是数字系统唯一认识的代码。但数值越大，位数越多，读写不方便，容易出错！

3. 八进制

- ❑ 数字符号：0, 1, 2, 3, 4, 5, 6, 7
- ❑ 计数规则：逢八进一
- ❑ 基数：8
- ❑ 权： 8^i , i 是各数位的序号
- ❑ 八进制数用下标“O”表示。例如：

$$(N)_8 = \sum_{i=-m}^{n-1} k_i \times 8^i$$

$$(752.34)_O = 7 \times 8^2 + 5 \times 8^1 + 2 \times 8^0 + 3 \times 8^{-1} + 4 \times 8^{-2}$$

因为 $2^3=8$ ，因而三位二进制数可用一位八进制数表示。

Decimal notation

Octal notation

Binary notation

Hexadecimal notation

4. 十六进制

- 数字符号：0~9、A、B、C、D、E、F
- 计数规则：逢十六进一
- 基数：16
- 权： 16^i , i 是各个数位的序号。
- 十六进制数用下标“H”表示，例如：

$$(N)_{16} = \sum_{i=-m}^{n-1} k_i \times 16^i$$

$$\begin{aligned} (\text{BD2.3C})_{\text{H}} &= \text{B} \times 16^2 + \text{D} \times 16^1 + 2 \times 16^0 + 3 \times 16^{-1} + \text{C} \times 16^{-2} \\ &= 11 \times 16^2 + 13 \times 16^1 + 2 \times 16^0 + 3 \times 16^{-1} + 12 \times 16^{-2} \end{aligned}$$

因为 $2^4=16$ ，所以四位二进制数可用一位十六进制数表示。

在计算机应用系统中，二进制主要用于机器内部的数据处理，八进制和十六进制主要用于书写程序，十进制主要用于运算最终结果的输出。

1.1.3 常用数制转换

不同数制之间的转换方法有若干种。

1、非十进制数转换成十进制数；

采用按权展开相加法。具体步骤是，首先把非十进制数写成按位权展开的多项式，然后按十进制数的计数规则求其和。

$$\text{例1} \quad (2A.8)_H = (?)_D$$

$$\begin{aligned} \text{解} \quad (2A.8)_H &= 2 \times 16^1 + A \times 16^0 + 8 \times 16^{-1} \\ &= 32 + 10 + 0.5 = (42.5)_D \end{aligned}$$

数字转换: $(10101.11)_B = (\quad ? \quad)_D$

- ☐ A $(12.75)_D$
- ☐ B $(21.57)_D$
- ☐ C $(12.57)_D$
- ☒ D $(21.75)_D$

提交

例 2 $(165.2)_O=(?)_D$

解
$$\begin{aligned}(165.2)_O &= 1 \times 8^2 + 6 \times 8^1 + 5 \times 8^0 + 2 \times 8^{-1} \\ &= 64 + 48 + 5 + 0.25 = (117.25)_D\end{aligned}$$

例3 $(10101.11)_B=(?)_D$

解
$$\begin{aligned}(10101.11)_B &= 1 \times 2^4 + 0 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 \\ &\quad + 1 \times 2^0 + 1 \times 2^{-1} + 1 \times 2^{-2} \\ &= 16 + 0 + 4 + 0 + 1 + 0.5 + 0.25 = (21.75)_D\end{aligned}$$

2 十进制数转换成其它进制数

a、整数转换：除基数取余法。

把十进制整数 N 转换成 R 进制数的步骤如下： (1) 将 N 除以 R ，记下所得的商和余数。

(2) 将上一步所得的商再除以 R ，记下所得商和余数。

(3) 重复做第(2)步，直到商为0；

(4) 将各个余数转换成 R 进制的数码，并按照和运算过程相反的顺序把各个余数排列起来，即为 R 进制的数。

例 4 $(427)_D = (?)_H$

解

$16 \overline{) 427}$	余数	
$16 \overline{) 26} \dots\dots\dots$	$11=B$	最低位
$16 \overline{) 1} \dots\dots\dots$	$10=A$	
$0 \dots\dots\dots 1=1$		最高位

即 $(427)_D = (1AB)_H$

先得到的余数，写在靠近小数点的位置。

例 5 $(427)_D = (?)_O$

解 $8 \overline{) 427}$ 余数

$8 \overline{) 53} \dots\dots\dots 3$ 最低位

$8 \overline{) 6} \dots\dots\dots 5$

$0 \dots\dots\dots 6$ 最高位

即 $(427)_D = (653)_O$

先得到的余数，写在靠近小数点的位置。

例 6 $(11)_D = (?)_B$

解

$2 \overline{) 11}$	余数	
$2 \overline{) 5} \dots\dots\dots$	1	最低位
$2 \overline{) 2} \dots\dots\dots$	1	
$2 \overline{) 1} \dots\dots\dots$	0	
$0 \dots\dots\dots$	1	最高位

即 $(11)_D = (1011)_B$

数字转换: $(217)_D = (\quad ? \quad)_B$

- ☐ A $(10011011)_B$
- ☒ B $(11011001)_B$
- ☐ C $(11001101)_B$
- ☐ D $(11011011)_B$

提交

例：求 $(217)_{10} = (\quad)_2$

解： $\because$

2		<u>217</u>	余1	b_0
2		<u>108</u>	余0	b_1
2		<u>54</u>	余0	b_2
2		<u>27</u>	余1	b_3
2		<u>13</u>	余1	b_4
2		<u>6</u>	余0	b_5
2		<u>3</u>	余1	b_6
2		<u>1</u>	余1	b_7
		0		

$\therefore (217)_{10} = (11011001)_2$

b、纯小数转换：乘基数取整法

把十进制的纯小数 M 转换成 R 进制数的步骤如下：

- (1) 将 M 乘以 R ，记下整数部分。
- (2) 将上一步乘积中的小数部分再乘以 R ，记下整数部分。
- (3) 重复做第(2)步，直到小数部分为0或者满足精度要求为止。
- (4) 将各步求得的整数转换成 R 进制的数码，并按照和运算过程相同的顺序排列起来，即为所求的 R 进制数。

例 7 $(0.85)_D = (?)_H$

取整部分

解 $0.85 \times 16 = 13.6 \dots\dots\dots 13 = D$ 最高位

$0.6 \times 16 = 9.6 \dots\dots\dots 9 = 9$

$0.6 \times 16 = 9.6 \dots\dots\dots 9 = 9$

$\vdots$

最低位

即 $(0.85)_D = (0.D99\dots)_H$

先得到的整数部分，写在靠近小数点的位置。

取整部分

最高位

.....6

.....3

.....1

•

•

•

最低位

即

$$(631\dots)_0$$

先得到的整数部分，写在靠近小数点的位置。

数字转换： $(0.375)_D = (\quad ? \quad)_B$

- ☐ A $(0.11)_B$
- ☐ B $(0.101)_B$
- ☒ C $(0.011)_B$
- ☐ D $(0.1101)_B$

提交

例 9 $(11.375)_D = (?)_B$

解

$$\begin{array}{r} 2 \overline{) 11} \\ 2 \overline{) 5} \dots\dots\dots 1 \\ 2 \overline{) 2} \dots\dots\dots 1 \\ 2 \dots \overline{) 1} \dots\dots\dots 0 \\ 0 \dots\dots\dots 1 \end{array}$$

即 $(11)_D = (1011)_B$

$$0.375 \times 2 = 0.75$$

$$0.75 \times 2 = 1.5$$

$$0.5 \times 2 = 1.0$$

即 $(0.375)_D = (0.011)_B$

故 $(11.375)_D = (1011.011)_B$

问：怎样在数据转换中，保证转换的精度不变？

转换后的精度与转换前的精度在同一个数量级就可以了。

例如： $23.45_{10} \approx 10111.0111010_2$

$\therefore 23.45$ 精度是： 10^{-2} 。

10111.0111010_2 精度是： 2^{-7} 。

$$2^{-7} < 10^{-2} < 2^{-6}$$

$\therefore$ 可以认为转换后的精度满足要求。

Number system Conversions



How to do this kind of conversion by Windows calculator?

Example : Convert 23.45_{10} into a binary number.

The precision must be ensured.

∴ The precision of 23.45 is 10^{-2} . $2^{-7} < 10^{-2} < 2^{-6}$

∴ The binary precision should be 2^{-7} .

$$23.45_{10} \times 2^7 = 3001.6_{10} \approx 3002_{10} \quad (\text{Radix point shifts right})$$

Convert 3002_{10} to binary by Windows calculator. $3002_{10} = 101110111010_2$

$$101110111010_2 \div 2^7 = 10111.0111010_2 \quad (\text{Radix point shifts left})$$

$$\therefore 23.45_{10} \approx 10111.011101_2$$



Number system Conversions

➤ Conversion Between Octal and Decimal

八进制与十进制之间的转换

Use the same method as before.

Example: $21.15_{10} = (\textcolor{red}{25.115})_8$ Note: Ensure the precision $8^{-3} < 10^{-2} < 8^{-2}$

$12.45_8 = (\textcolor{red}{10.58})_{10}$ Note: Ensure the precision $10^{-2} < 8^{-2} < 10^{-1}$

➤ Conversion Between Hex and Decimal

十六进制与十进制之间的转换

Use the same method as before.

Example: $24.36_{10} = (\textcolor{red}{18.5C})_{16}$ Note: Ensure the precision $16^{-2} < 10^{-2} < 16^{-1}$

$18.5B_{16} = (\textcolor{red}{24.355})_{10}$ Note: Ensure the precision $10^{-3} < 16^{-2} < 10^{-2}$

Number system Conversions

Exercise 1:

Convert 0.39_{10} into a **binary number**. The precision must achieve 10%. (误差小于10%)

$$\because 2^{-4} < 10\% < 2^{-3}$$

$\therefore$ The binary precision should be 2^{-4} .

Exercise 2:

Convert 0.39_{10} into an **octal number**. The precision must achieve 10%. (误差小于10%)

$$\because 8^{-2} < 10\% < 8^{-1}$$

$\therefore$ The octal precision should be 8^{-2} .

3 二进制数转换成八进制数或十六进制数

三位二进制数对应一位八进制数。四位二进制数对应一位十六进制数。

二进制数转换成八进制数(或十六进制数)时，其整数部分和小数部分可以同时进行转换。

其方法是：以二进制数的小数点为起点，分别向左、向右，每三位(或四位)分一组。

对于小数部分，最低位一组不足三位(或四位)时，必须在有效位右边补0，使其足位。然后，把每一组二进制数转换成八进制(或十六进制)数，并保持原排序。

例10 $(1011011111.10011)_B = (\quad ? \quad)_O = (\quad ? \quad)_H$

解

1011011111.100110											
1	3	3	7	4	6						

所以 $(1011011111.100110)_B = (1337.46)_O$

1011011111.10011000											
2	D	F	.	9	8						

即 $(1011011111.10011)_B = (2DF.98)_H$

4 八进制数或十六进制数转换成二进制数

八进制(或十六进制)数转换成二进制数时， 只要把八进制(或十六进制)数的每一位数码分别转换成三位(或四位)的二进制数， 并保持原排序即可。

例11 $(36.24)_O = (?)_B$

解 $(36.24)_O = (\underbrace{011}_{3}\underbrace{110}_{6}.\underbrace{010}_{2}\underbrace{100}_{4})_B = (11110.0101)_B$

例 12 $(3DB.46)_H = (?)_B$

解 $(3DB.46)_H = (\underbrace{001}_{3}\underbrace{111}_{D}\underbrace{101}_{B}.\underbrace{01000}_{4}\underbrace{110}_{6})_B$
 $= (1111011011.0100011)_B$

General Positional-Number-System Conversion

- ★ Use Hexadecimal as the middle state between Binary and Decimal.

$$(0010\ 1100\ 0110)_2 = (\quad\quad\quad)_{10}$$

$$(0010\ 1100\ 0110)_2 = (2\ C\ 6)_{16} = (710)_{10}$$

$$(1000\ 1)_{10} = (\quad\quad\quad)_2$$

$$(10001)_{10} = (2711)_{16} = (0010\ 0111\ 0001\ 0001)_2$$

不同进制数的转换

(1) 将R进制数转换成十进制数:

规则: 只要将R进制数按位权展开, 再按十进制运算规则运算, 即可得到十进制数。

(2) 将十进制数转换成R进制数:

规则: 需将十进制数的整数部分和小数部分分别进行转换, 然后将它们合并起来。整数部分转换时, 用除R取余法。
小数部分转换时, 用乘R取整法。

对于将十进制数转换成二进制数,

整数部分转换时, 用除2取余法, 倒记余数。

小数部分转换时, 用乘2取整法, 正向取整。

(3) 基数R为 2^k 各进制之间的互相转换

表1-1 几种计数进制数的对照表

十进制	二进制	八进制	十六进制
0	0000	0	0
1	0001	1	1
2	0010	2	2
3	0011	3	3
4	0100	4	4
5	0101	5	5
6	0110	6	6
7	0111	7	7
8	1000	10	8
9	1001	11	9
10	1010	12	A
11	1011	13	B
12	1100	14	C
13	1101	15	D
14	1110	16	E
15	1111	17	F

Arithmetic Operations

算术运算

➤ Binary Addition (二进制加运算)

Be similar to decimal.

Take one byte for example:

$$X = 122_{10} = 01111010_2$$

$$Y = 92_{10} = 01011100_2$$

$$S = 214_{10} = 11010110_2$$

0	1	1	1	1	0	0	0	0	C
	0	1	1	1	1	0	1	0	X
+	0	1	0	1	1	1	0	0	Y
	1	1	0	1	0	1	1	0	S

C: carry

S: sum

二进制数中逢2进1

Arithmetic Operations

➤ Binary Subtraction (二进制减运算)

Be similar to decimal.

Take one byte for example:

$$X = 122_{10} = 01111010_2$$

$$Y = 92_{10} = 01011100_2$$

$$D = 30_{10} = 00011110_2$$

0	0	0	1	1	1	0	0	0	B
	0	1	1	1	1	0	1	0	X
-	0	1	0	1	1	1	0	0	Y
	0	0	0	1	1	1	1	0	D

B: borrow

D: difference

二进制数中高位借1相当于低位的2。

Arithmetic Operations

➤ Octal Addition

(八进制数的加运算)

Be similar to decimal.

Take one byte for example:

$$X = 122_{10} = 172_8$$

$$Y = 92_{10} = 134_8$$

$$S = 214_{10} = 326_8$$

0	1	0	0	C
	1	7	2	X
+	1	3	4	Y
	3	2	6	S

八进制数中逢8进1

C: carry **S: sum**

Arithmetic Operations

➤ Octal Subtraction

(八进制数的减运算)

Be similar to decimal.

Take one byte for example:

$$X = 122_{10} = 172_8$$

$$Y = 92_{10} = 134_8$$

$$D = 30_{10} = 36_8$$

0	0	1	0	B
	1	7	2	X
-	1	3	4	Y
	0	3	6	D

八进制数中高
位借1相当于低
位的8。

B: borrow

D: difference

Arithmetic Operations

➤ Hex Addition

(十六进制数的加运算)

Be similar to decimal.

Take one byte for example:

$$X = 122_{10} = 7A_{16}$$

$$Y = 92_{10} = 5C_{16}$$

$$S = 214_{10} = D6_{16}$$

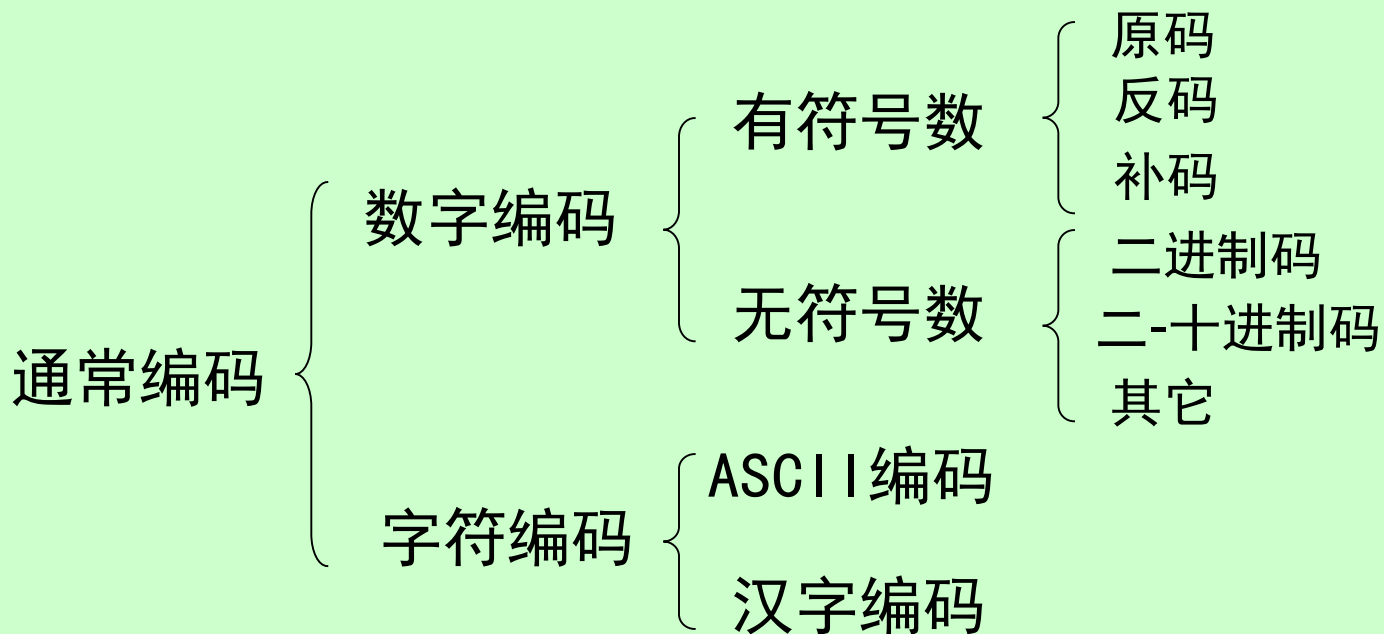
	0	1	0	C
		7	A	X
+		5	C	Y
		D	6	S

十六进制数中逢16进1

C: carry **S: sum**

1. 4 二进制编码

给一个信息或符号指定一个具体的二进制码去代表它，这一过程称为二进制编码



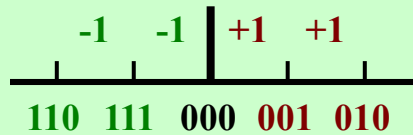
有符号数（原码）的表示

➤ Signed-Magnitude Representation

Usually, the MSB is used as the sign bit.

0: plus, 1: minus

0:正号 1: 负号



Take 4-bit number for example:

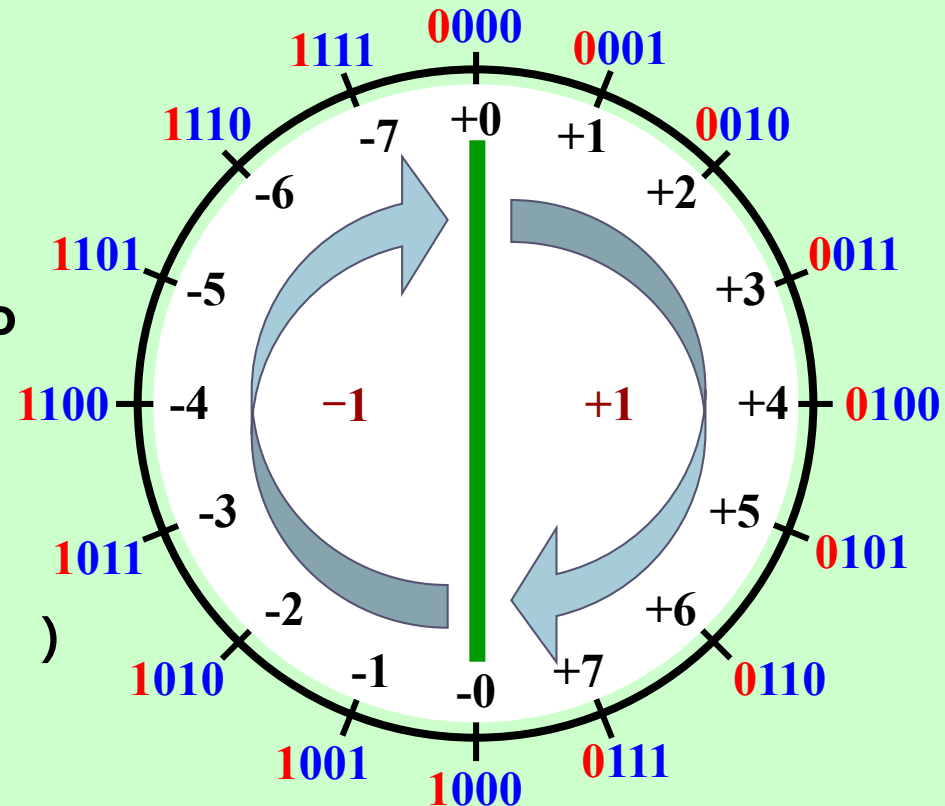
以4位bit数为例

A signed-magnitude number is also called a **True Code** (原码).

It has an equal number of positive and negative integers. (直接在数值位的前面加上一个bit符号位。)

An n -bit integer range is :

$$-(2^{n-1} - 1) \sim +(2^{n-1} - 1)$$



图中红色的bit位为符号位

有符号数（原码）的表示

➤ Signed-Magnitude Representation

Take 8-bit numbers for example:

$$A = \textcolor{red}{0}101\ 1101_2 = +93_{10}$$

$$-A = \textcolor{red}{1}101\ 1101_2 = -93_{10}$$

$$B = \textcolor{red}{1}110\ 1011_2 = -107_{10}$$

$$-B = \textcolor{red}{0}110\ 1011_2 = +107_{10}$$

上面的原码中，红色的bit位为符号位，符号位后面的都是数值位。

Advantage of Signed-Magnitude :

- Negating a number is very easy.
 - Just need to change its sign.

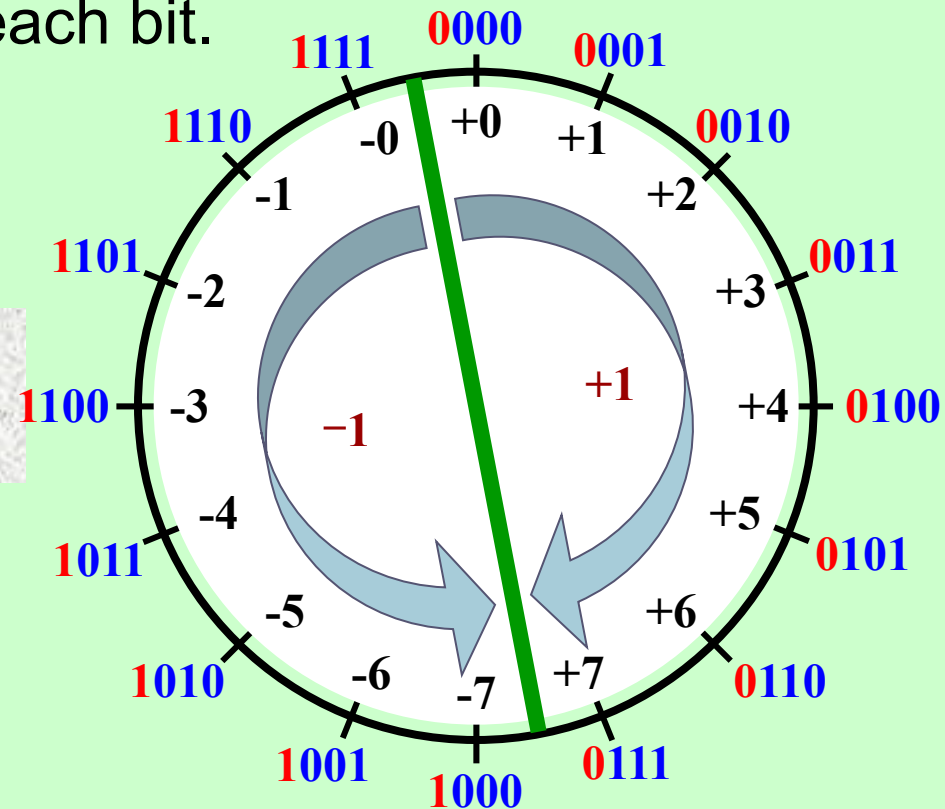
反码

➤ Ones'-Complement Representation —— Bitwise Inversion (按位求反)

Negate a number by inverting each bit.

Take 4-bit number for example:
(以4bit 位数为例)

Can we use *Ones' complement* to represent negative integers?



反码

➤ Ones'-Complement Representation —— Bitwise Inversion (按位求反)

Negate a number by inverting each bit.

Take 4-bit number for example:

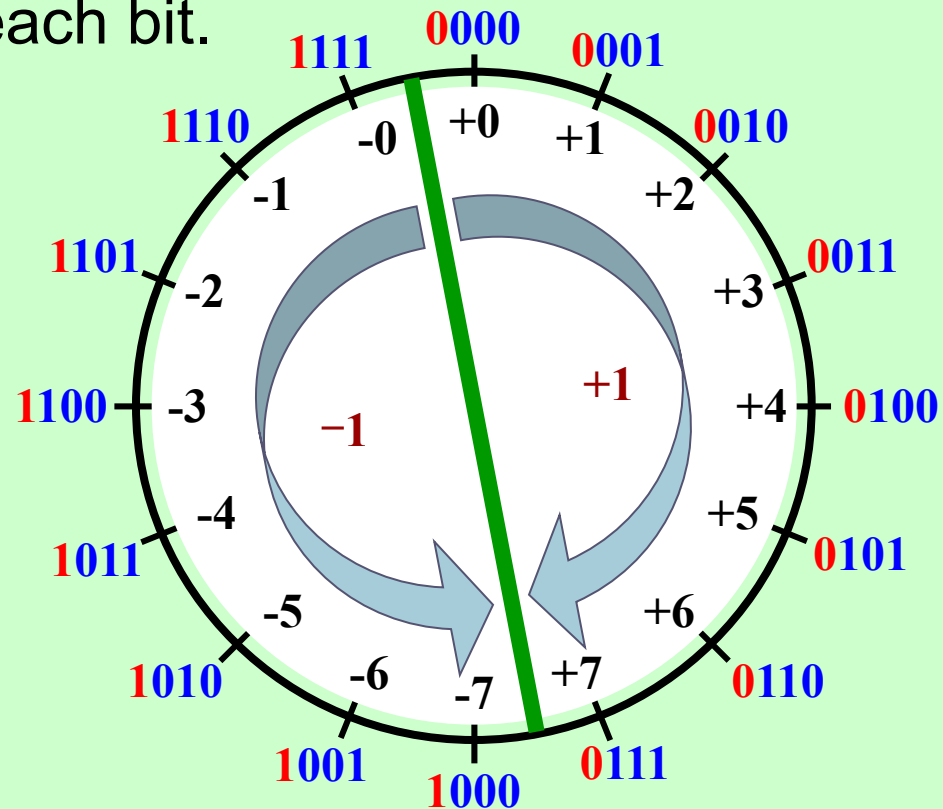
(以4bit 位数为例)

It has an equal number of positive and negative integers.

The MSB can also be used as the sign bit.
0 = plus , 1 = minus

An n-bit integer range is :

$$-(2^{n-1} - 1) \sim +(2^{n-1} - 1)$$



补码

补码又称为对2的补码（2's complement）。用补码表示有符号数时，符号位保持不变，数值大小定义为

$$(N)_{\text{补码}} = \begin{cases} N & (\text{正数时}) \\ 2^n - N & (\text{负数时}) \end{cases}$$

正数的原码、反码和补码形式相同。求负数的补码时，为了避免做减法运算，一般方法是：先求出负数的反码（将原码的数值位逐位求反），然后在最低位加1即可得到补码，即

$$(N)_{\text{补码}} = (N)_{\text{反码}} + 1 \quad (\text{负数时})$$

问题：怎样由补码得到原码？

$$((N)_{\text{补码}})_{\text{补码}} = (N)_{\text{原码}}$$

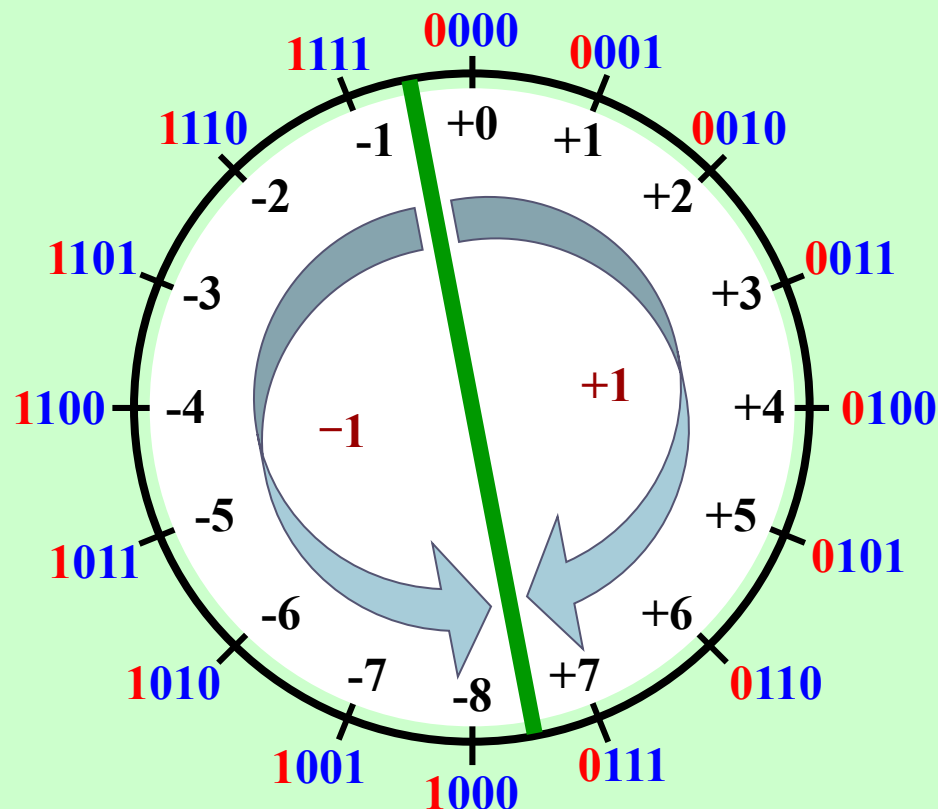
补码

Take 4-bit number for example:

(以4bit 位数为例)

The MSB can also be used as the sign bit.
0 = plus , 1 = minus

An n-bit integer range is :
 $-(2^{n-1}) \sim +(2^{n-1} - 1)$



若 $(N)_{\text{补码}}=1000$ 求 $(N)_{\text{原码}}=?$

解答：由于符号位为1，所以N必为负数。其数值位的求法如下：
将数值位000对应位求反，然后加1，得到数值部分值为1000，也就是8，所以1000是-8的补码

补码

6位有符号二进制数系统里：

Example of Addition:

有了补码就可以很容易的实现正数与负数的运算了。

0	0	0	1	0	0	Carry
	0	0	0	1	0	+2
+	0	0	0	1	1	+3
	0	0	1	0	1	+5

Correct!

1	1	1	1	1	0	Carry
	1	1	0	1	1	-5
+	1	1	1	0	1	-3
	1	1	0	0	0	-8

Correct!

0	0	0	0	1	0	Carry
	0	0	1	0	1	+5
+	1	1	0	0	1	-7
	1	1	1	1	0	-2

Correct!

A negative integer is just the **Complement** of its positive integer !

$$A - B = A + \text{Module} - B = A + (-B)$$

∴ Subtractor is not necessary !

1.4 二—十进制码进制码

（BCD码——Binary Code Decimal）用4位二进制数来表示一位十进制数中的0~9十个数码。

4位二进制数有16种组合，可从这16种组合中选择10种组合分别来表示十进制的0~9十个数。选哪10种组合，有多种方案，这就形成了不同的BCD码。

几种常见的BCD码

0000
0001
0010
0011
0100
0101
0110
0111
1000
1001
1010
1011
1100
1101
1110
1111

十进制数	有权码			无权码	
	8421码	2421码	5421码	余3码	余3循环码
0	0000	0000	0000	0011	0010
1	0001	0001	0001	0100	0110
2	0010	0010	0010	0101	0111
3	0011	0011	0011	0110	0101
4	0100	0100	0100	0111	0100
5	0101	1011	1000	1000	1100
6	0110	1100	1001	1001	1101
7	0111	1101	1010	1010	1111
8	1000	1110	1011	1011	1110
9	1001	1111	1100	1100	1010

几种常见的BCD码

0000
0001
0010
0011
0100
0101
0110
0111
1000
1001
1010
1011
1100
1101
1110
1111

十进制数	有权码			无权码	
	8421码	2421码	5421码	余3码	余3循环码
0	0000	0000	0000	0011	0010
1	0001	0001	0001	0100	0110
2	0010	0010	0010	0101	0111
3	0011	0011	0011	0110	0101
4	0100	0100	0100	0111	0100
5	0101	1011	1000	1000	1100
6	0110	1100	1001	1001	1101
7	0111	1101	1010	1010	1111
8	1000	1110	1011	1011	1110
9	1001	1111	1100	1100	1010

几种常见的BCD码

0000
0001
0010
0011
0100
0101
0110
0111
1000
1001
1010
1011
1100
1101
1110
1111

十进制数	有权码			无权码	
	8421码	2421码	5421码	余3码	余3循环码
0	0000	0000	0000	0011	0010
1	0001	0001	0001	0100	0110
2	0010	0010	0010	0101	0111
3	0011	0011	0011	0110	0101
4	0100	0100	0100	0111	0100
5	0101	1011	1000	1000	1100
6	0110	1100	1001	1001	1101
7	0111	1101	1010	1010	1111
8	1000	1110	1011	1011	1110
9	1001	1111	1100	1100	1010

几种常见的BCD码

0000
0001
0010
0011
0100
0101
0110
0111
1000
1001
1010
1011
1100
1101
1110
1111

十进制数	有权码			无权码	
	8421码	2421码	5421码	余3码	余3循环码
0	0000	0000	0000	0011	0010
1	0001	0001	0001	0100	0110
2	0010	0010	0010	0101	0111
3	0011	0011	0011	0110	0101
4	0100	0100	0100	0111	0100
5	0101	1011	1000	1000	1100
6	0110	1100	1001	1001	1101
7	0111	1101	1010	1010	1111
8	1000	1110	1011	1011	1110
9	1001	1111	1100	1100	1010

各种编码的特点

- ◆ 有权码：编码与所表示的十进制数之间的转换容易。

$$(1001\ 0000)_{8421BCD} = (90)_D$$

- ◆ 余3码：0和9,1和8.....6和4的余3码互为反码，这对于求取对10的补码很方便。如将两个余3码相加的和是十进制的10时，正好是二进制的16，于是可从高位自动产生进位信号。
- ◆ 余3码循环码：相邻的两个代码之间仅一位的状态不同。按余3码循环码组成计数器时，每次转换过程只有一个触发器翻转，译码时不会发生竞争一冒险现象。

用BCD代码表示十进制数

对于一个多位的十进制数，需要有与十进制位数相同的几组BCD代码来表示。例如：

$$\begin{aligned} (463.5)_{10} &= \left[\begin{array}{c} \text{0100} \quad \text{0110} \quad \text{0011} \cdot \text{0101} \\ 4 \quad 6 \quad 3 \quad 5 \end{array} \right]_{8421\text{BCD}} \\ (863.2)_{10} &= \left[\begin{array}{c} \text{1110} \quad \text{1100} \quad \text{0011} \cdot \text{0010} \\ 8 \quad 6 \quad 3 \quad 2 \end{array} \right]_{2421\text{BCD}} \end{aligned}$$

不能省略！

不能省略！

求BCD代码表示的十进制数

对于有权BCD码，可以根据位权展开求得所代表的十进制数。例如：

$$(0111)_{8421\text{BCD}} = 0 \times 8 + 1 \times 4 + 1 \times 2 + 1 \times 1 = (7)_{10}$$

$$(1101)_{2421\text{BCD}} = 1 \times 2 + 1 \times 4 + 0 \times 2 + 1 \times 1 = (7)_{10}$$

格雷码

- ◆ 格雷码是一种无权码。
- ◆ 编码特点是：任何**两个相邻代码之间仅有一位不同**。
- ◆ 该特点常用于模拟量的转换。当模拟量发生微小变化，而引起数字量发生变化时，格雷码仅仅改变一位，这与其它码同时改变2位或更多的情况相比，更加可靠。

二进制码 $b_3b_2b_1b_0$	格雷码 $G_3G_2G_1G_0$	二进制码 $b_3b_2b_1b_0$	格雷码 $G_3G_2G_1G_0$
0 0 0 0	0 0 0 0	1 0 0 0	1 1 0 0
0 0 0 1	0 0 0 1	1 0 0 1	1 1 0 1
0 0 1 0	0 0 1 1	1 0 1 0	1 1 1 1
0 0 1 1	0 0 1 0	1 0 1 1	1 1 1 0
0 1 0 0	0 1 1 0	1 1 0 0	1 0 1 0
0 1 0 1	0 1 1 1	1 1 0 1	1 0 1 1
0 1 1 0	0 1 0 1	1 1 1 0	1 0 0 1
0 1 1 1	0 1 0 0	1 1 1 1	1 0 0 0

Gray Code

格雷码

Gray Code Constructing

构造格雷码-----异或运算实现

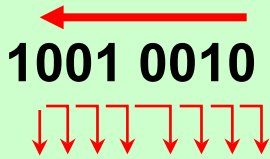
From binary number to Gray code

The width is same, the MSB is same;

From **right to left**, if **a bit in binary number is same as its right bit**, the **gray bit is 0**. If it is different, **the gray bit is 1**.

Examples:

Binary number: 1001 0010
Gray codes: 1101 1011



二进制码 1001 0010
格雷码 1101 1011



Gray Code

Translation between Gray Code and Binary Code

Binary code: $b_{n-1} b_{n-2} b_{n-3} \dots b_1 b_0$

Gray code: $g_{n-1} g_{n-2} g_{n-3} \dots g_1 g_0$

$$g_i = b_{i+1} \oplus b_i \quad g_{n-1} = b_{n-1} \oplus 0 = b_{n-1} \quad (n-1 \text{ 是最高位})$$

5的二进制码是: 0101 $\longrightarrow$ 5的格雷码是: 0111

$$\because A = B \oplus C \quad \longrightarrow \quad C = A \oplus B \quad \longrightarrow \quad B = A \oplus C$$

$$\therefore b_i = b_{i+1} \oplus g_i \quad b_{n-1} = g_{n-1} \oplus 0 = g_{n-1} \quad (n-1 \text{ 是最高位})$$

7的格雷码是: 0100 $\longrightarrow$ 7的二进制码是: 0111

奇偶校验码

信息位 + 1位校验位。

具有检错能力，能发现奇数个代码位同时出错的情况。

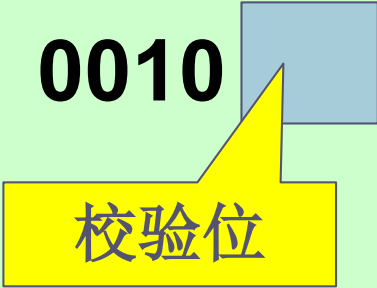
构成：信息位（可以是任一种二进制代码）及一位校验位。

校验位数码的编码方式：

“奇校验”时，使校验位和信息位所组成的每组代码中含有奇数个1；

“偶校验”时，使校验位和信息位所组成的每组代码中含有偶数个1。

二进制码1001 0010



校验位

完